

Student Handbook

Docker Training

Day 1 · Session 2

Writing Dockerfiles for .NET Web API & Angular

10:45 – 12:15 · 1.5 hours · Hands-on Lab

Prerequisites: Completed Day 1 Session 1 — Docker is installed and running

Learning objectives

By the end of this session you will be able to:

- Explain every Dockerfile instruction (FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, ENTRYPOINT, CMD)
- Write a single-stage Dockerfile and understand its limitations
- Write a multi-stage Dockerfile for .NET 8 Web API with a dramatically smaller output image
- Write a multi-stage Dockerfile for Angular using Node to build and Nginx to serve
- Use .dockerignore to exclude build artefacts and speed up builds
- Understand how layer caching works and how to order instructions to exploit it
- Compare Alpine vs Debian base images and choose the right one for your use case

Dockerfile instructions — reference

A Dockerfile is a plain text file with one instruction per line. Docker reads it top to bottom, executing each instruction as a new read-only layer on top of the previous one.

Instruction	What it does
<code>FROM <image>:<tag></code>	Sets the base image for all subsequent instructions. Must be the first instruction.
<code>WORKDIR /path</code>	Sets (and creates if needed) the working directory for RUN, COPY, ENTRYPOINT, CMD.
<code>COPY <src> <dest></code>	Copies files from the build context (your machine) into the image.
<code>RUN <command></code>	Executes a shell command at build time and commits the result as a new layer.
<code>ENV KEY=value</code>	Sets an environment variable visible at both build time and runtime.
<code>ARG NAME=default</code>	Build-time variable only. Not available in the final image or at runtime.
<code>EXPOSE <port></code>	Documents which port the container listens on. Does NOT actually publish the port.
<code>ENTRYPOINT ["cmd"]</code>	The executable to run when the container starts. Hard to override.
<code>CMD ["arg"]</code>	Default arguments passed to ENTRYPOINT. Easily overridden with <code>docker run <image> <cmd></code> .
<code>USER <name></code>	Switches to a non-root user for all subsequent RUN / CMD / ENTRYPOINT instructions.
<code>.dockerignore</code>	Not an instruction — a file in the build context that excludes paths from COPY.

How layer caching works

Docker caches the result of each instruction. On a rebuild, if an instruction and everything above it are unchanged, Docker reuses the cached layer — making rebuilds very fast.

⚡ The golden rule of caching

Copy things that change LAST. If you copy your source code before installing dependencies, every code change busts the dependency cache and forces a full npm install or dotnet restore.

Cache-friendly order for a .NET project:

- COPY *.csproj first — project file changes rarely
- RUN dotnet restore — cached as long as .csproj hasn't changed
- COPY . . — copy all source code
- RUN dotnet publish — runs only when source actually changed

Cache-friendly order for an Angular project:

- COPY package.json package-lock.json first — lockfile changes rarely
- RUN npm ci — cached as long as lockfile hasn't changed
- COPY . . — copy all source
- RUN ng build — runs only when source changed

Multi-stage builds

A multi-stage build uses multiple FROM instructions in one Dockerfile. Each FROM starts a fresh stage. You selectively COPY artefacts from one stage into the next — leaving behind compilers, SDKs, node_modules, and anything not needed at runtime.

✗ Single-stage .NET image

Includes the full .NET SDK (compilers, MSBuild, templates). Final image ≈ 850 MB — shipped to production!

✓ Multi-stage .NET image

Stage 1 uses sdk:8.0 to compile. Stage 2 uses aspnet:8.0 (runtime only). Final image ≈ 220 MB — only the app and runtime.

Hands-on lab steps

1

Create the .NET 8 Web API project

Scaffold a minimal project you will containerise throughout this session

Create a new Web API project

```
# Create a working directory for today's labs
mkdir docker-training && cd docker-training

# Scaffold a new .NET 8 Web API
dotnet new webapi -n WeatherApi --no-openapi
cd WeatherApi

# Run it locally first to confirm it works
dotnet run
# Visit https://localhost:5001/weatherforecast in your browser
# Press Ctrl+C to stop
```

Explore the project structure

```
ls -la
# You should see: WeatherApi.csproj Program.cs Controllers/
```

Note

The goal is to containerise this exact project. You are not changing any application code in this session — only the Docker configuration around it.

2

Write a single-stage Dockerfile for .NET (and see the problem)

Understand why single-stage is not suitable for production

Create Dockerfile.single in the WeatherApi folder

```
# Create the file
touch Dockerfile.single

# Open it in your editor and paste the content below
```

Dockerfile.single — content

```
FROM mcr.microsoft.com/dotnet/sdk:8.0
WORKDIR /app

# Copy project file first (cache the restore step)
COPY WeatherApi.csproj .
RUN dotnet restore

# Copy all source code
COPY . .

# Build and publish
RUN dotnet publish -c Release -o /app/publish

EXPOSE 8080
ENTRYPOINT ["dotnet", "/app/publish/WeatherApi.dll"]
```

Build and check the image size

```
docker build -f Dockerfile.single -t weatherapi:single .
```

```
docker images weatherapi
```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
weatherapi	single	alb2c3d4e5f6	8 seconds ago	~850MB

What is wrong here?

The final image contains the entire .NET SDK — compilers, MSBuild, templates, NuGet cache. None of that is needed to run the app. You are shipping 630 MB of build tooling to every production server.

3 Write a multi-stage Dockerfile for .NET (production-ready)

Stage 1 compiles with the SDK. Stage 2 runs with the lightweight runtime.

Create Dockerfile in the WeatherApi folder

```
touch Dockerfile
# Open it in your editor and paste the content below
```

Dockerfile — full content

```
# — Stage 1: Build —————
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copy project file first so dotnet restore is cached separately
COPY WeatherApi.csproj .
RUN dotnet restore

# Copy remaining source and publish
COPY . .
RUN dotnet publish -c Release -o /app/publish --no-restore

# — Stage 2: Runtime —————
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# Copy ONLY the published output from the build stage
COPY --from=build /app/publish .

# Run as non-root for security (covered in depth on Day 4)
RUN adduser --disabled-password --gecos '' appuser && chown -R appuser /app
USER appuser

EXPOSE 8080
ENTRYPOINT ["dotnet", "WeatherApi.dll"]
```

Build the multi-stage image

```
docker build -t weatherapi:1.0 .

# Compare sizes side by side
docker images weatherapi
```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
weatherapi	1.0	b2c3d4e5f6a7	12 seconds ago	~220MB
weatherapi	single	alb2c3d4e5f6	5 minutes ago	~850MB

Run and test the multi-stage image

```
docker run -d --name api -p 8080:8080 weatherapi:1.0

# Test the endpoint
curl http://localhost:8080/weatherforecast

# Or open http://localhost:8080/weatherforecast in your browser
```

✓ Result

The production image is ~220 MB — 630 MB smaller — and contains only the ASP.NET runtime and your compiled app. No SDK. No source code. No build tools.

4

Create .dockerignore — speed up builds and reduce context size

Tell Docker what NOT to send to the daemon during builds

Every time you run `docker build`, Docker sends the entire build context (the folder you specify) to the daemon. Without `.dockerignore`, this includes `bin/`, `obj/`, and `.git/` — often hundreds of megabytes of files that are never used.

Create .dockerignore in the WeatherApi folder

```
# Create the file
touch .dockerignore
# Open in your editor and paste:
```

.dockerignore — content

```
# Build output — always generated fresh inside the container
bin/
obj/

# Git history — never needed in an image
.git/
.gitignore

# Docker files themselves
Dockerfile*
.dockerignore

# Editor and OS files
.vs/
.vscode/
*.user
.DS_Store

# Test projects (don't ship tests to production)
**/*.Tests/
**/*.Test/
```

Verify the context size difference

```
# Remove old image, rebuild clean – watch the 'Sending build context' line
docker rmi weatherapi:1.0
docker build -t weatherapi:1.0 .
```

Expected output:

```
# Without .dockerignore you might see:
=> [internal] load build context   2.3s
=> => transferring context: 85.2MB

# With .dockerignore:
=> [internal] load build context   0.1s
=> => transferring context: 12.3kB
```

⚡ Why this matters

A smaller build context means faster builds, especially on CI servers where the context is uploaded over a network. It also prevents accidentally leaking secrets or credentials that live in your project folder.

5 Write a multi-stage Dockerfile for Angular

Node builds the app. Nginx serves the static files. The result is tiny.

Create the Angular project (from the docker-training root)

```
cd .. # back to docker-training/

# Create a new Angular app (skip git init)
npx @angular/cli@latest new weather-ui --routing=false --style=css --skip-git
cd weather-ui
```

Create Dockerfile in the weather-ui folder

```
touch Dockerfile
# Paste the content below
```

Dockerfile — full content

```
# — Stage 1: Build —————
FROM node:20-alpine AS build
WORKDIR /app

# Copy lockfiles first – npm ci is cached until these change
COPY package.json package-lock.json .
RUN npm ci

# Copy source and build for production
COPY . .
RUN npx ng build --configuration production

# — Stage 2: Serve —————
FROM nginx:1.27-alpine AS runtime

# Remove default Nginx page
```

```

RUN rm -rf /usr/share/nginx/html/*

# Copy Angular build output
COPY --from=build /app/dist/weather-ui/browser /usr/share/nginx/html

# Optional: custom nginx config (see tip below)
# COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Create .dockerignore for Angular

```

# .dockerignore
node_modules/
dist/
.git/
.gitignore
Dockerfile*
.dockerignore
.angular/
.vscode/
*.md

```

Build and run the Angular image

```

docker build -t weather-ui:1.0 .

# Compare sizes
docker images weather-ui

```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
weather-ui	1.0	c3d4e5f6a7b8	15 seconds ago	~45MB

```

# Run the UI on port 4200
docker run -d --name ui -p 4200:80 weather-ui:1.0

# Open http://localhost:4200 - you should see the Angular welcome page

```

What just happened

node_modules (~300 MB) and the Angular compiler were used in Stage 1 and then discarded. The final image is Nginx + your compiled JS/HTML/CSS — only ~45 MB.

6

Demonstrate layer caching in action

Change only the source code and watch what gets re-used

This step shows caching in a concrete, measurable way.

First build — everything runs from scratch

```

# In weather-ui/ - time the first build
time docker build -t weather-ui:cache-test .

```


Expected output:

```
=> [build 2/5] COPY package.json package-lock.json .    0.1s
=> [build 3/5] RUN npm ci                                42.3s ← slow
=> [build 4/5] COPY . .                                  0.2s
=> [build 5/5] RUN npx ng build --configuration prod    18.1s ← slow

real    1m2.4s
```

Change only a source file — then rebuild

```
# Simulate a source code change
echo "<!-- updated -->" >> src/app/app.component.html

# Rebuild
time docker build -t weather-ui:cache-test .
```

Expected output:

```
=> CACHED [build 2/5] COPY package.json package-lock.json .    0.0s
=> CACHED [build 3/5] RUN npm ci                                0.0s ← cached!
=> [build 4/5] COPY . .                                  0.2s
=> [build 5/5] RUN npx ng build --configuration prod          17.8s

real    0m18.3s
```

⚡ Result

42 seconds saved on npm ci because the package files didn't change. In a real project with hundreds of packages, this is the difference between a 3-minute CI build and a 30-second one.

Alpine vs Debian base images

Factor	Alpine (alpine, -alpine)	Debian (bookworm-slim, etc.)
Base image size	~5 MB	~70–100 MB
Package manager	apk	apt-get
Shell available	sh (not bash by default)	bash
.NET compatibility	Use mcr.microsoft.com/dotnet/aspnet:8.0-alpine	Use aspnet:8.0 (default)
Node compatibility	node:20-alpine ☒ Works well	node:20 (larger)
Common issues	Missing glibc — some native npm packages fail	Very compatible — rarely breaks
Best for	Node/Angular, simple Go/Python apps	.NET, apps with native dependencies

Quick reference

Command	What it does
<code>docker build -t name:tag .</code>	Build image from Dockerfile in current directory
<code>docker build -f Dockerfile.dev -t name:tag .</code>	Build using a specific Dockerfile name
<code>docker build --no-cache -t name:tag .</code>	Force rebuild — ignore all cached layers
<code>docker images</code>	List all local images with sizes
<code>docker rmi name:tag</code>	Remove a local image
<code>docker image prune</code>	Remove all dangling (untagged) images
<code>docker history name:tag</code>	Show all layers and their sizes
<code>docker run -d --name x -p HOST:CONT img</code>	Run container in background with port map
<code>docker rm -f name</code>	Force stop and remove a container

Common mistakes to avoid

✗ Mistake 1 — Copying source before restoring packages

`COPY . .` before `RUN dotnet restore` or `RUN npm ci` means every code change forces a full package restore. Always copy only the package manifest first, restore, then copy source.

✗ Mistake 2 — No `.dockerignore`

Without `.dockerignore`, the build context includes `node_modules/` (hundreds of MB), `bin/`, `obj/`, and `.git/` — all sent to the Docker daemon on every build. Always add `.dockerignore` before your first build.

✗ Mistake 3 — Using the tag 'latest'

`FROM node:latest` changes silently when the upstream releases a new version. Pin to a specific tag like `node:20-alpine` to make builds reproducible.

✗ Mistake 4 — Forgetting `--from=build` in multi-stage `COPY`

`COPY /app/publish .` copies from the build context (your machine). `COPY --from=build /app/publish .` copies from the earlier build stage. Always include `--from=` in multi-stage Dockerfiles.